Experiment.5

Aim:Simulation and interfacing

Theory:

The concept of building an **obstacle-avoiding car** using **Arduino** and an **ultrasonic sensor** involves creating a robot car that can autonomously detect obstacles in its path and avoid them. The car will use an ultrasonic sensor to measure the distance to nearby objects, and based on the data from this sensor, the Arduino microcontroller will process the information to make decisions and control the car's movement.

The key components for this project are:

1. **Arduino Microcontroller**: This is the heart of the system that controls the car's movements based on sensor inputs.
2. **Ultrasonic Sensor (HC-SR04)**: This sensor measures the distance between the sensor and obstacles. It works on the principle of sound waves and provides real-time distance data.
3. **DC Motors**: These motors are used to drive the wheels of the car.
4. **Motor Driver (L298N or L293D)**: This IC controls the speed and direction of the motors based on signals from the Arduino.
5. **Power Supply**: A battery or power supply to provide the necessary energy for the motors and Arduino.

Ultrasonic Sensor (HC-SR04) Working:

The ultrasonic sensor emits high-frequency sound waves from a transmitter. These sound waves bounce off obstacles and return to the sensor. The time it takes for the sound waves to return is measured by the sensor. Using the speed of sound in air, the distance to the obstacle can be calculated with the following formula:

Distance=Time×Speed of Sound2\text{Distance} = \frac{\text{Time} \times \text{Speed of Sound}}{2}Distance=2Time×Speed of Sound

Where the time is the duration between emitting the sound wave and receiving the echo, and the speed of sound is typically 343 meters per second at room temperature. Dividing by 2 accounts for the round trip of the sound wave.

Arduino-Controlled Obstacle-Avoiding Car:

The Arduino uses the data from the ultrasonic sensor to make decisions about how the car should move. The basic logic of the system can be outlined as follows:

1. **Initialize and Setup**:
    - The Arduino initializes the ultrasonic sensor, motors, and motor driver.
    - It also sets the motor control pins as outputs.
2. **Obstacle Detection**:
    - The Arduino continuously reads the distance from the ultrasonic sensor.
    - If an obstacle is detected within a certain threshold distance (e.g., 20 cm), the car must take action.
3. **Movement Control**:
    - **Move Forward**: If no obstacle is detected within the threshold, the car moves forward.
    - **Turn Left/Right**: If an obstacle is detected in the front, the car will rotate to the left or right to avoid the obstacle.
    - **Reverse**: In some cases, the car may reverse slightly before turning to avoid getting too close to an obstacle.
    - **Stop**: If an obstacle is too close (e.g., 5 cm), the car might stop to avoid a collision.
4. **Motor Driver and Control**:
    - The motor driver (e.g., L298N) receives signals from the Arduino to control the rotation and speed of the motors. The direction of the motor is changed by sending high or low signals to the motor driver pins connected to the motors.
    - The car's direction is controlled by sending signals to the motor driver to rotate left, right, forward, or reverse.
5. **Simulation of Obstacle-Avoidance Logic**:
    - In a simulated environment, this process can be tested and visualized before physical implementation. The simulation can model the movement of the car and obstacle interactions based on sensor inputs and motor control logic.

Simulation:

In a simulation environment like Tinkercad or Proteus, you can simulate the entire system before building the physical car. Here’s a general flow of steps for simulation:

1. **Set up Arduino and Components**: In the simulation tool, you connect the ultrasonic sensor, motors, and motor driver to the Arduino board.
2. **Write Code**: You write the Arduino code to control the movement of the car based on ultrasonic sensor data.
3. **Run Simulation**: The simulation runs, and you can test how the car moves when it detects obstacles.
4. **Debug and Refine**: You can make adjustments to the code and hardware connections as necessary, using the simulation results to refine your design.

Interfacing Components:

1. **Arduino to Ultrasonic Sensor**:
    - The **trigger** pin of the ultrasonic sensor is connected to a digital output pin on the Arduino.
    - The **echo** pin is connected to a digital input pin on the Arduino.
    - The Arduino sends a pulse to the trigger pin, and then listens for the echo pin to receive the response after the sound wave is reflected back from the obstacle.
2. **Arduino to Motor Driver (L298N)**:
    - The **IN1**, **IN2**, **IN3**, and **IN4** pins of the motor driver are connected to digital output pins on the Arduino for controlling motor direction.
    - The **ENA** and **ENB** pins control the motor speed, and these can be connected to PWM pins on the Arduino to allow speed control.
    - The motor driver's output pins are connected to the motors that will drive the wheels of the car.
3. **Motor Driver to Motors**:
    - The motor driver directly controls the motors, allowing the car to move forward, backward, left, or right.

Arduino Code Example:

```
//including the libraries

#include <AFMotor.h>

#include <NewPing.h>

#include <Servo.h>


//defining pins and variables

#define TRIG_PIN A4

#define ECHO_PIN A5

#define MAX_DISTANCE 200

#define MAX_SPEED 200 // sets speed of DC motors

#define MAX_SPEED_OFFSET 20


#define turn_amount 500


//defining motors,servo,sensor

NewPing sonar(TRIG_PIN, ECHO_PIN, MAX_DISTANCE);

AF_DCMotor motor1(2, MOTOR12_8KHZ);
```

```
AF_DCMotor motor2(1,
MOTOR12_8KHZ);

Servo myservo;


//defining global variables

boolean goesForward=false;

int distance = 100;

int speedSet = 0;


void setup() {
  Serial.begin(9600);
  myservo.attach(10);
  myservo.write(90);
  delay(2000);
  distance = readPing();
  delay(100);
  distance = readPing();
  delay(100);
  distance = readPing();
  delay(100);
  distance = readPing();
  delay(100);
}
```

```
void loop() {
 int distanceR = 0;
 int distanceL = 0;
 delay(40);
 Serial.println(distance);


 if(distance<=15)
 {
  Serial.println("Object Detected");
  moveStop();
  delay(100);
  moveBackward();
  delay(300);
  moveStop();
  delay(200);
  distanceR = lookRight();
  Serial.print("Distance Right = ");
  Serial.println(distanceR);
  delay(200);
  distanceL = lookLeft();
  Serial.print("Distance Left = ");
  Serial.println(distanceL);
  delay(200);
```

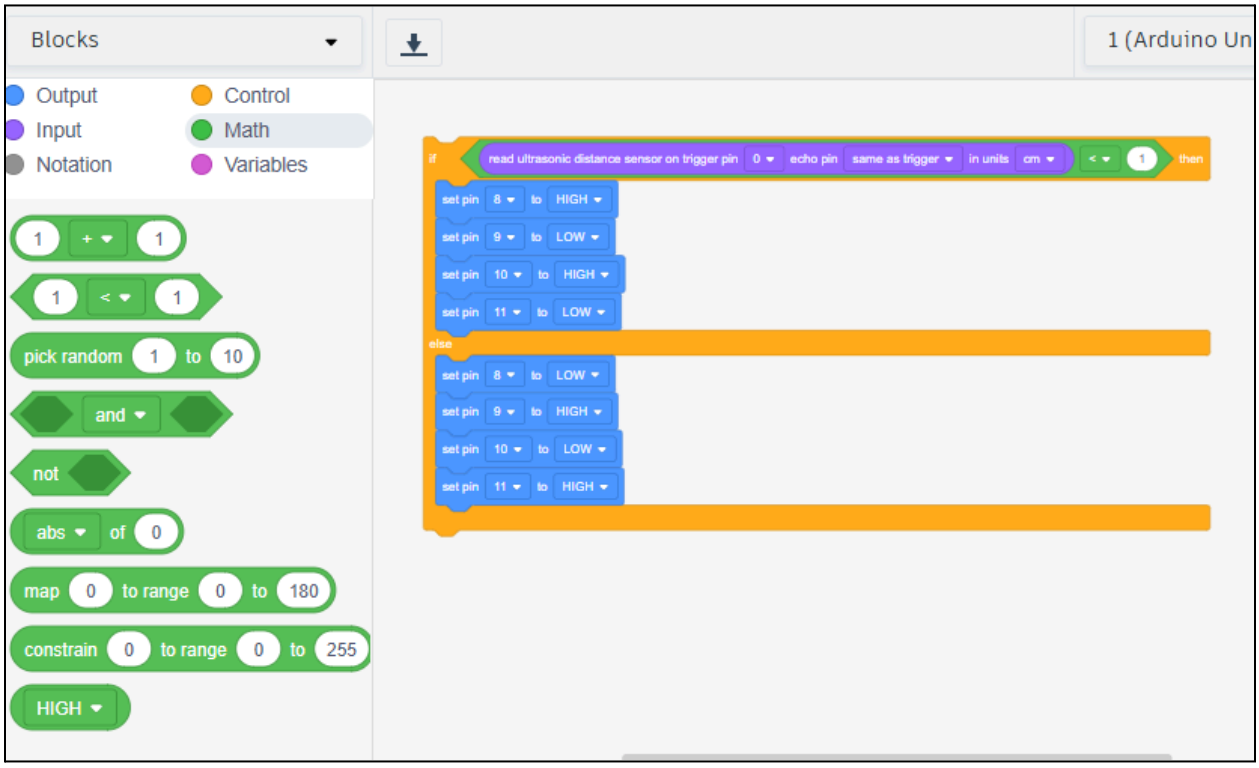
```
 if(distanceR>=distanceL)
 {
  turnRight();
  moveStop();
 }
 else
 {
  turnLeft();
  moveStop();
 }
}
else
{
 moveForward();
}

//reseting the variable after the operations
distance = readPing();
}

int lookRight()
{
   myservo.write(0);
   delay(500);
   int distance = readPing();
   delay(100);
   myservo.write(90);
   return distance;
}

int lookLeft()
{
   myservo.write(180);
   delay(500);
   int distance = readPing();
   delay(100);
   myservo.write(90);
   return distance;
   delay(100);
}

int readPing() {
  delay(70);
  int cm = sonar.ping_cm();
  if(cm==0)
```

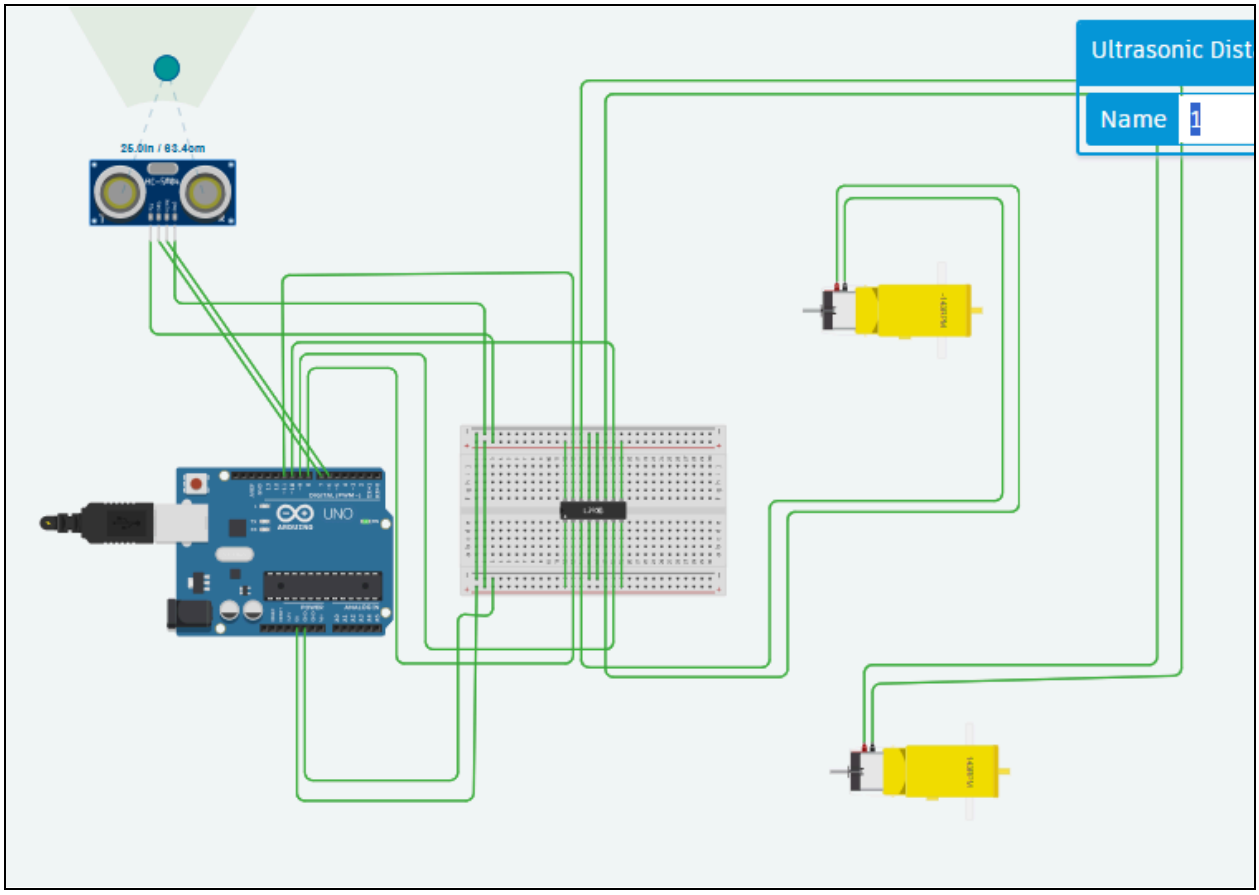
```
  {
    cm = 250;
  }
  return cm;
}

void moveStop() {
  motor1.run(RELEASE);
  motor2.run(RELEASE);
  }

void moveForward() {

 if(!goesForward)
  {
    goesForward=true;
    motor1.run(FORWARD);
    motor2.run(FORWARD);
   for (speedSet = 0; speedSet < MAX_SPEED; speedSet +=2) // slowly bring the speed up to avoid loading down the batteries too quickly
   {
    motor1.setSpeed(speedSet);
motor2.setSpeed(speedSet+MAX_SPEED_OFFSET);
    delay(5);
   }
  }
}

void moveBackward() {
    goesForward=false;
    motor1.run(BACKWARD);
    motor2.run(BACKWARD);
  for (speedSet = 0; speedSet < MAX_SPEED; speedSet +=2) // slowly bring the speed up to avoid loading down the batteries too quickly
  {
    motor1.setSpeed(speedSet);
motor2.setSpeed(speedSet+MAX_SPEED_OFFSET);
    delay(5);
  }
}

void turnRight() {
```

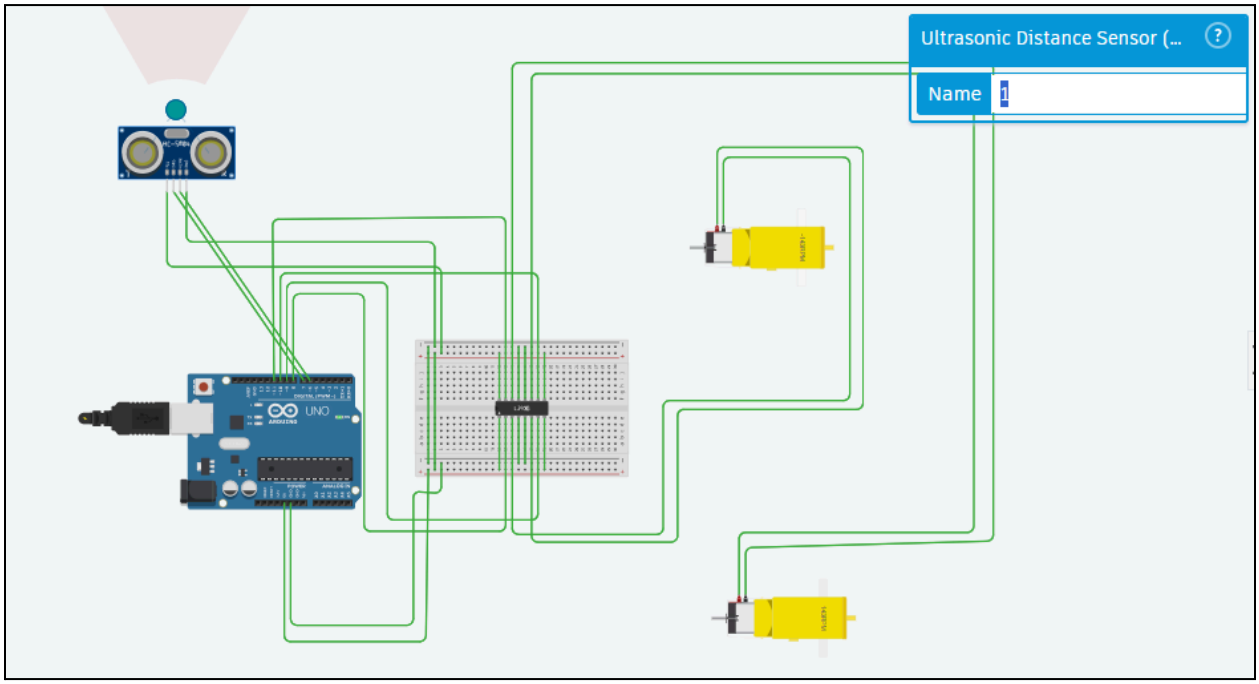
```
  Serial.println("Turning Right");
  motor1.run(FORWARD);
  motor2.run(BACKWARD);
  delay(turn_amount);
  motor1.run(FORWARD);
  motor2.run(FORWARD);
}
```

```
void turnLeft() {
  Serial.println("Turning Left");
  motor1.run(BACKWARD);
  motor2.run(FORWARD);
  delay(turn_amount);
  motor1.run(FORWARD);
  motor2.run(FORWARD);
}
```

Output:




Ultrasonic Dist
Name
1
25.0in / 63.4cm
UNO


Ultrasonic Distance Sensor (...
Name
1
UNO

Conclusion

The obstacle-avoiding car successfully demonstrates autonomous navigation using an Arduino and ultrasonic sensor. By detecting obstacles and adjusting movement accordingly, the car efficiently avoids collisions. The project highlights key concepts in sensor interfacing, motor control, and real-time decision-making, offering practical insights into embedded systems and robotics.